

ModelViewer Ray Tracing Display Mode

Consolidated Design Document

Combined from prior design notes and discussions; redundancy removed and pseudo-code retained where useful.

1. Executive Summary

This document consolidates the ray tracing design discussion for ModelViewer into a single, non-redundant design specification. It covers the recommended architecture, camera implications, Embree strategy, RTX clarification, scene snapshot approach, progressive rendering pipeline, threading model, pseudo-code, risk areas and implementation roadmap.

Recommendation: Add ray tracing as a progressive display mode, not as a replacement for the current OpenGL renderer.

The preferred near-term strategy is an Embree-backed CPU progressive path tracer that renders into an offscreen buffer and presents the result through the existing Qt/OpenGL viewport. This preserves the current raster renderer while enabling high-quality ray traced visualization. The design must stay backend-neutral so that OptiX or Vulkan Ray Tracing can be added later without rewriting the scene extraction, material mapping, camera, or accumulation logic.

2. Goals and Non-goals

2.1 Goals

- Introduce RayTracedPreview and PathTracedFinal display modes.
- Reuse existing scene, material, camera, environment and floor data wherever practical.
- Maintain semantic parity between raster PBR and ray traced output.
- Keep OpenGL as the presentation layer for the first implementation.
- Use immutable scene snapshots to isolate the ray tracer from live scene graph mutations.
- Use progressive accumulation and worker threads to keep the viewport responsive.
- Adopt Embree as the first production-quality acceleration backend after the correctness prototype.
- Keep the architecture ready for a future OptiX or Vulkan RT backend.

2.2 Non-goals for the First Release

- Full renderer migration from OpenGL to Vulkan.
- OptiX-only or NVIDIA-only architecture.
- Full physically complete material parity with all glTF extensions.
- Transparency, refraction, subsurface scattering, clearcoat and anisotropy in the first milestone.
- Real-time game-style ray tracing as the initial target.
- Complex denoising pipeline in the first working version.

3. Current ModelViewer Context

ModelViewer currently has a Qt/OpenGL-based viewport and an increasingly modular rendering architecture with controller separation. The ray tracing design should fit into this direction and avoid pushing ray tracing logic into the viewport widget itself.

- Qt viewport based on QOpenGLWidget or equivalent OpenGL presentation.
- OpenGL raster renderer with PBR-like material evaluation.
- Material parameters such as albedo/base color, metalness and roughness.
- Image-based lighting and a pseudo-floor setup for visual realism.
- SceneRuntime, SceneRenderController and ViewportInteractionController style separation.
- Support for mesh-based imported models such as GLTF/GLB and OBJ.

4. Camera Update and Ray Tracing Impact

A key update is that the previous turntable-camera shortcut has been removed. The viewer now uses a proper camera model where the view vector is calculated from the actual camera position and fragment position.

```
// Raster shader view vector
vec3 V = normalize(cameraPos - fragPos);
```

This simplifies ray tracing substantially because primary rays can be generated using the same camera parameters as raster rendering. Orbit, pan, zoom, perspective projection, reflection vectors and environment lookups become naturally aligned between raster and ray traced modes.

5. Proposed Architecture

Ray tracing should be another renderer backend controlled by SceneRenderController. The viewport widget should not become the owner of ray tracing logic.

```
RenderWidget / ViewportWidget
|
v
SceneRenderController
|
+-- OpenGLRasterRenderer
|
+-- RayTracingRenderer
    |
    +-- CpuPathTracer
    +-- IRayIntersector
        |
        +-- NaiveCpuIntersector // initial correctness/debug backend
        +-- EmbreeIntersector  // recommended production CPU backend
        +-- OptixIntersector    // future NVIDIA RTX backend
        +-- VulkanRtBackend     // future GPU-native backend
```

5.1 Display Modes

```
enum class DisplayMode
```

```
{
    RasterPBR,
    Wireframe,
    Normals,
    RayTracedPreview,
    PathTracedFinal
};
```

5.2 Renderer Interface

```
class IViewportRenderer
{
public:
    virtual ~IViewportRenderer() = default;

    virtual void initialize() = 0;
    virtual void resize(int width, int height) = 0;
    virtual void setSceneSnapshot(std::shared_ptr<const RtSceneSnapshot> scene) = 0;
    virtual void renderFrame(const RenderFrameContext& context) = 0;
    virtual void shutdown() = 0;
};

class OpenGLRasterRenderer : public IViewportRenderer {};
class RayTracingRenderer : public IViewportRenderer {};
```

6. Backend Strategy

Backend	Effort	Expected Performance	Vendor Dependency	Role in ModelViewer
Custom CPU BVH / naïve traversal	Low	Low to Medium	None	Debug and correctness prototype only
Embree CPU	Medium	High	None	Recommended first production backend
OptiX	High	Very High	NVIDIA	Future RTX acceleration path
Vulkan Ray Tracing	Very High	Very High	Low / cross-vendor	Long-term GPU-native architecture option

6.1 Embree Clarification

Embree is the right near-term backend for ModelViewer because it gives high-performance ray intersection and acceleration structures without forcing a graphics API migration. However, Embree should not be treated as a transparent NVIDIA RTX path. For NVIDIA RTX RT core acceleration, the future backend should be OptiX or Vulkan Ray Tracing.

6.2 Recommended Backend Evolution

1. First: implement a simple internal intersector or direct triangle traversal for camera/material/debug validation.
2. Second: introduce IRayIntersector and move all hit/occlusion queries behind the interface.
3. Third: implement EmbreeIntersector as the default production backend.

- Fourth: evaluate OptiX for RTX acceleration while preserving the same camera, scene snapshot, material and accumulation design.
- Fifth: evaluate Vulkan RT only if a full GPU-native renderer becomes a strategic requirement.

7. Scene Snapshot Design

The ray tracer should consume immutable scene snapshots. This is one of the most important design choices because it prevents worker threads from reading half-updated scene data and creates a stable contract between ModelViewer runtime and the rendering backend.

```
struct RtSceneSnapshot
{
    std::vector<RtMesh> meshes;
    std::vector<RtInstance> instances;
    std::vector<RtMaterial> materials;

    RtEnvironment environment;
    RtCamera camera;
    RtFloor floor;

    uint64_t revisionId = 0;
};
```

7.1 Mesh and Instance Representation

```
struct RtMesh
{
    std::vector<glm::vec3> positions;
    std::vector<glm::vec3> normals;
    std::vector<glm::vec2> texcoords;
    std::vector<uint32_t> indices;
};

struct RtInstance
{
    uint32_t meshId;
    uint32_t materialId;

    glm::mat4 localToWorld;
    glm::mat4 worldToLocal;
};
```

This separation is intentionally similar to BLAS/TLAS thinking: mesh geometry is reusable, while instances carry transforms and material bindings. This will keep the architecture compatible with Embree, OptiX and Vulkan RT style acceleration structures.

7.2 Scene Builder

```
class RtSceneBuilder
{
public:
    std::shared_ptr<RtSceneSnapshot> buildFromSceneRuntime(
        const SceneRuntime& runtime,
```

```
    const CameraSnapshot& camera);  
};
```

8. Camera Snapshot and Ray Generation

Raster and ray traced modes should use a shared camera snapshot. The ray tracing camera can then be derived from that snapshot.

```
struct CameraSnapshot  
{  
    glm::vec3 position;  
    glm::vec3 target;  
    glm::vec3 up;  
  
    float fovY;  
    float nearPlane;  
    float farPlane;  
  
    glm::mat4 viewMatrix;  
    glm::mat4 projectionMatrix;  
};
```

```
struct RtCamera  
{  
    glm::vec3 position;  
    glm::vec3 forward;  
    glm::vec3 right;  
    glm::vec3 up;  
  
    float verticalFovRadians;  
    float aspectRatio;  
};
```

```
Ray generatePrimaryRay(float pixelX, float pixelY)  
{  
    float ndcX = (2.0f * pixelX / width) - 1.0f;  
    float ndcY = 1.0f - (2.0f * pixelY / height);  
  
    float tanHalfFov = std::tan(camera.verticalFovRadians * 0.5f);  
  
    glm::vec3 dir = glm::normalize(  
        camera.forward +  
        ndcX * camera.aspectRatio * tanHalfFov * camera.right +  
        ndcY * tanHalfFov * camera.up);  
  
    return Ray{ camera.position, dir };  
}
```

9. Material Mapping

The initial ray traced material should mirror the current raster PBR vocabulary. This keeps output understandable and avoids introducing a second material language.

```
struct RtMaterial
{
    glm::vec3 baseColor = glm::vec3(0.8f);
    float metalness = 0.0f;
    float roughness = 0.5f;

    glm::vec3 emissiveColor = glm::vec3(0.0f);
    float emissiveStrength = 0.0f;

    float opacity = 1.0f;
};
```

9.1 Initial Shading Targets

- Environment color on miss.
- Base color and normal visualization.
- Diffuse term for non-metallic surfaces.
- Fresnel-Schlick and roughness-controlled reflection.
- Metalness blending between diffuse and specular response.
- Pseudo-floor reflection and contact shadow as viewer-specific enhancements.

10. Rendering Flow

```
User selects RayTracedPreview
    |
    v
SceneRenderController changes display mode
    |
    v
RtSceneBuilder creates immutable RtSceneSnapshot
    |
    v
RayTracingRenderer receives snapshot and resets accumulation
    |
    v
Worker threads progressively render samples
    |
    v
Latest frame buffer is uploaded as OpenGL texture
    |
    v
Viewport displays progressively refined image
```

10.1 Frame Buffer and Presentation

```
struct RtFrameBuffer
{
    int width = 0;
```

```

    int height = 0;

    std::vector<float> rgba32f;    // Linear HDR RGBA
    uint32_t sampleCount = 0;
};

class RtTexturePresenter
{
public:
    void initialize();
    void resize(int width, int height);
    void upload(const RtFrameBuffer& frame);
    void drawFullscreen();
    void shutdown();
};

```

11. Progressive Accumulation

```

struct RtFrameAccumulator
{
    int width = 0;
    int height = 0;

    std::vector<glm::vec4> accumulatedRadiance;
    uint32_t sampleCount = 0;

    void reset();
    void accumulateSample(int x, int y, const glm::vec3& color);
    glm::vec4 resolvePixel(int x, int y) const;
};

```

```

if (sceneRevisionChanged || cameraChanged || resizeChanged)
{
    rayTracer.resetAccumulation();
}

rayTracer.renderNextSamples(sampleBudgetPerFrame);
displayAccumulatedImage();

```

Condition	Behavior
Camera moving	Use low sample count or partial tile update; prioritize interactivity
Camera stopped	Continue accumulation until preview/final sample target
Geometry changed	Rebuild acceleration structure and reset accumulation
Material changed	Reset accumulation; rebuild only material data if geometry unchanged
Environment/floor changed	Reset accumulation
Viewport resized	Resize buffers and reset accumulation

12. Threading and Cancellation Model

The ray tracer should not run synchronously inside `paintGL()`. UI interaction and OpenGL presentation must remain responsive.

```
UI Thread:
    handle input
    paintGL()
    upload latest completed RT buffer
    draw fullscreen texture
```

```
RT Worker Thread(s):
    wait for active scene snapshot
    render tiles progressively
    accumulate samples
    publish latest frame buffer
```

```
std::atomic<uint64_t> activeRevision;
std::atomic_bool cancelRequested;

// On camera, material, environment, floor or scene change:
activeRevision++;
cancelRequested = true;
rayTracer.resetAccumulation();
startNewRenderJob();
```

13. Core Class Skeletons

13.1 Ray Tracing Renderer

```
class RayTracingRenderer
{
public:
    void initialize();
    void resize(int width, int height);
    void setSceneSnapshot(std::shared_ptr<const RtSceneSnapshot> scene);
    void renderFrame(const RenderFrameContext& context);
    void resetAccumulation();
    void shutdown();

private:
    std::unique_ptr<CpuPathTracer> m_pathTracer;
    std::unique_ptr<RtTexturePresenter> m_presenter;
    std::shared_ptr<const RtSceneSnapshot> m_scene;
};
```

13.2 Ray Intersector Interface

```
class IRayIntersector
{
public:
    virtual ~IRayIntersector() = default;
```



```

    virtual bool intersect(const Ray& ray, Hit& hit) const = 0;
    virtual bool occluded(const Ray& ray) const = 0;
};

class NaiveCpuIntersector : public IRayIntersector {};
class EmbreeIntersector   : public IRayIntersector {};

```

13.3 Embree Intersector Skeleton

```

class EmbreeIntersector : public IRayIntersector
{
public:
    bool build(const RtSceneSnapshot& scene);

    bool intersect(const Ray& ray, Hit& hit) const override;
    bool occluded(const Ray& ray) const override;

private:
    RTCDevice m_device = nullptr;
    RTCScene  m_scene  = nullptr;
};

```

14. Dirty Flags and Invalidation Rules

```

enum class RtDirtyFlag : uint32_t
{
    None          = 0,
    Camera        = 1 << 0,
    Geometry      = 1 << 1,
    Transform     = 1 << 2,
    Material      = 1 << 3,
    Environment   = 1 << 4,
    Floor         = 1 << 5,
    Viewport      = 1 << 6
};

```

Dirty Flag	Action
Camera	Reset accumulation only
Geometry	Rebuild acceleration structure; reset accumulation
Transform	Update/rebuild instances; reset accumulation
Material	Update material buffer; reset accumulation
Environment	Reset accumulation
Floor	Reset accumulation
Viewport	Resize buffers; reset accumulation

15. Implementation Roadmap

Milestone	Focus	Deliverable
1	Renderer boundary	RayTracedPreview mode displays a generated test image

2	Camera rays	Environment gradient or IBL lookup follows the camera correctly
3	Scene snapshot extraction	Meshes, instances, transforms and materials available to RT backend
4	First geometry hit	Model silhouette, normal debug and depth debug work
5	Basic shading	Base color and simple environment/reflection shading
6	Progressive accumulation	Image refines while camera is idle; resets on changes
7	Embree integration	Embree-backed intersect/occluded queries replace prototype traversal
8	Floor and environment	Pseudo-floor reflection, contact shadow/AO and IBL alignment
9	Quality modes	Preview and final sample count controls
10	Future GPU evaluation	OptiX/RTX feasibility and Vulkan RT long-term evaluation

16. Risks and Mitigations

Risk	Mitigation
Ray tracing slows down UI	Run rendering on worker threads; UI only presents latest buffer
Camera mismatch between raster and RT	Use shared CameraSnapshot for both paths
Thread safety issues	Use immutable RtSceneSnapshot and revision-based cancellation
Material mismatch	Derive RtMaterial from existing material data
Large assembly performance	Use Embree acceleration and instancing-aware scene layout
Scope creep into renderer rewrite	Keep OpenGL presentation and avoid Vulkan in phase one
Incorrect RTX expectation from Embree	Separate Embree CPU backend from future OptiX/Vulkan RTX backend
Noisy preview	Use progressive accumulation and optional denoising later

17. Final Direction

The most practical and robust direction for ModelViewer is:

- Keep OpenGL as the real-time raster and presentation pipeline.
- Add ray tracing as a separate progressive display mode.
- Use immutable RtSceneSnapshot as the bridge from ModelViewer runtime to ray tracing.
- Validate correctness with a simple CPU path first.
- Adopt Embree as the production CPU intersection backend.
- Preserve the same architecture for future OptiX RTX and Vulkan RT backends.

18. Consolidated Design Decisions

- Ray tracing is a display mode, not a replacement renderer.
- The updated regular camera model enables clean, standard primary ray generation.
- Scene access should be snapshot-based and immutable during rendering.
- Renderer code should stay behind SceneRenderController and not inside the viewport widget.
- Embree is the recommended initial production backend, but not an NVIDIA RTX substitute.
- OptiX is the preferred future NVIDIA RTX acceleration path.
- Vulkan Ray Tracing is deferred until/if a GPU-native rendering architecture becomes necessary.
- Progressive accumulation and cancellation are mandatory for good CAD-viewer usability.
- Pseudo-floor and environment rendering should be integrated after the basic ray tracing path is stable.